ChatGPT

执行摘要

过去五年中，“长思维链”相关研究迅速发展，主要聚焦于如何让大规模语言模型（LLM）在复杂推理任务中生成更长、更可靠的推理链条，并突破上下文长度和记忆限制。早期工作如Wei等人（2022）提出的链式思维提示（Chain-of-Thought, CoT）在算术、常识推理等任务中大幅提升性能【38†L23-L31】；其后出现的自洽性（Self-Consistency）策略通过采样多条推理路径并投票选答案，使链式推理效果进一步提高【41†L25-L33】【41†L31-L35】。针对CoT难以泛化到更难任务的问题，Zhou等人（2022）提出“最简到复杂”策略（Least-to-Most），先解决简单子问题再逐步递进，显著改善了组合推理能力【44†L25-L34】【44†L35-L43】。与此同时，工具调用结合链式推理的框架（如ReAct【48†L27-L35】【48†L36-L43】、Toolformer【53†L29-L37】、PAL【69†L46-L54】【69†L55-L60】等）使模型能够在推理中调用计算器、搜索引擎、代码解释器等外部工具，有效缓解了纯内生推理的错误累计和计算瓶颈。近年来，研究者还引入基于搜索的推理（如“思想树”Tree-of-Thoughts【61†L27-L35】【61†L36-L40】）和自学习强化策略（OpenAI o1、DeepSeek-R1等）以探索多条思路并自我改进。此外，记忆增强模型（如MemoryLLM【29†L24-L32】、M+【28†L21-L30】【28†L33-L36】）通过在Transformer中引入潜空间记忆和检索机制，将可处理上下文长度从万级扩展到十万级。本文系统梳理上述重要工作：对每篇论文给出发表信息、问题定义、方法架构与关键技术（如记忆机制、分层推理、检索增强、稀疏注意、分段训练等）、实验设置与结果对比、优缺点及开源代码情况；并通过汇总表格对比各方法的模块类型、复杂度、可扩展性、计算/内存成本、效果提升和适用序列长度等维度。最后，我们分析现有研究的共性和瓶颈，如梯度/记忆衰减、上下文窗口限制、推理一致性问题及标注评估难点，指出未充分探索或存在矛盾之处，并给出至少五个可行的创新方向（含组合创新）的思路、挑战和评估指标建议。同时使用Mermaid图展示方法演进时间线和模块依赖关系，以期为长推理链研究提供全面、系统的参考。

方法详表

方法/论文 (年份/会议)	核心模块与技术细节	数据集与定量结果 (相对基线)	开源代码/复现难度
Wei 等 (2022，NeurIPS)【38†L23-L31】 **Chain-of-Thought**	**问题：** 利用CoT提示提取中间推理步骤提升复杂推理能力 **架构：** 仅提示引导的自回归模型，无新增模块 **关键点：** 基于大模型（540B），提供示例推理链，可处理算术、常识、符号推理等任务【38†L23-L31】	GSM8K算术题：8个CoT示例下，540B模型达成最新SOTA，超过微调GPT-3+验证器【38†L29-L34】。其他任务如CommonsenseQA、SVAMP、ARC等也显著提升。	开源微小：仅提示设计，无需修改模型；适用于有连贯步骤任务，但对模型规模依赖大（需要百亿级参数）。
Wang 等 (2022, ICLR)【41†L25-L33】【41†L31-L35】 **Self-Consistency**	**问题：** 针对CoT贪婪解码易受单一路径误差影响的问题 **架构：** 无额外模型结构，改变解码策略 **关键点：** 多次采样生成多条推理路径，再边缘化答案；假设题目唯一答案下不同思路多次出现频率最高【41†L25-L33】	在GSM8K（算术）、SVAMP、AQuA、StrategyQA、ARC-Challenge等基准上，Self-Consistency相对于单条CoT提升巨大：GSM8K+17.9%、SVAMP+11.0%等【41†L31-L35】。	简单可复现：只改变抽样与投票策略，无新增参数；需额外计算成本（多条采样）。适用于需要多样化思路、答案唯一的推理任务。

方法/论文 (年份/会议)	核心模块与技术细节	数据集与定量结果 (相对基线)	开源代码/复现难度
Zhou 等 (2022, ICLR)【44†L25-L34】【44†L35-L43】 **Least-to-Most (L2M)**	**问题：** CoT难以泛化到比提示示例更难的任务 **架构：** 提示级策略，无模型改动 **关键点：** 将复杂问题分解为若干依赖关系的子问题，按易到难顺序依次求解，每步使用先前答案。【44†L25-L34】	在组合符号、数学任务（如SCAN语言转换等）上效果显著；以GPT-3 code-davinci-002为例，对SCAN任何数据划分均≥99%正确率，而常规模型仅16%【44†L35-L43】。说明L2M实现了超出示例难度的泛化。	可复现性高：仅提示设计；需要设计正确的分解策略，对问题结构理解依赖较强。适用于可分解的问题链路（如符号推理、编程题）。
Gao 等 (2022, arXiv)【69†L46-L54】【69†L55-L60】 **Program-Aided Language Model (PAL)**	**问题：** LLM虽能分步骤理解问题，但在计算/逻辑求解上易出错 **架构：** 让LLM生成可执行程序（如Python），并调用解释器完成数值计算 **关键点：** LLM负责问题理解和分解，计算交给程序。支持数学、符号和算法推理。	在13个数学和算法任务上，PAL+Codex达到SOTA。GSM8K上PAL (Codex) 击败PaLM-540B CoT，准确率提升15%以上【69†L55-L60】。其他BIG-Bench Hard子集也显著领先。	依赖程序解释器：需将问题翻译为代码，对LLM编程能力要求高。代码执行可避免数值误差，但程序生成错误会影响结果。代码/数据集公开（http://reasonwithpal.com）。
Yao 等 (2023, ICLR)【48†L27-L35】【48†L36-L43】 **ReAct: 推理与行动结合**	**问题：** CoT仅推理不行动，无法借助外部信息；纯依赖内部易产生错误 **架构：** 在生成过程中交替产生“思考（Observation）”与“行动（Action）”——如搜索、API调用 **关键点：** 推理帮助构建行动计划并处理异常，行动则通过外部工具获取附加信息；交互式决策框架。【48†L27-L35】	在HotpotQA、多跳Fact验证(FEVER)等QA任务中，ReAct通过维基API减少了CoT的误导和错误传递，性能优于基线【48†L33-L40】。在ALFWorld/WebShop等交互环境任务，ReAct在仅1-2示例下比传统模仿或RL方法高出34%、10%成功率【48†L36-L43】。	需提供或创建环境/工具接口，增加计算和延迟成本（API调用等）。效果好，且推理轨迹可解释性好。部分代码开源【48†L41-L44】。
Schick 等 (2023, NeurIPS)【53†L29-L37】 **Toolformer**	**问题：** LLM对基础功能（算术、检索等）不擅长；大模型才能自如使用 **架构：** 在预训练后，用自监督方式教模型学习在何时调用哪些API工具 **关键点：** 模型学习插入调用标记，自主决定使用计算器、QA系统、搜索引擎、翻译器、日历等工具，对标记填参后续融入生成。【53†L29-L37】	无需示例，仅通过少量工具示例自监督训练。Toolformer在多种零样本任务（如算术、问答、翻译、常识QA等）上表现大幅提升，经常可与参数更多的模型竞争。【53†L33-L37】	模型需要针对每种工具进行自监督训练；推理时有工具调用的额外开销。能显著提高基础运算和事实查找能力，代码公开（见NeurIPS 2023）。

方法/论文 (年份/会议)	核心模块与技术细节	数据集与定量结果 (相对基线)	开源代码/复现难度
Yao 等 (2023, NeurIPS) 【61†L27-L35】【61†L36-L40】 **Tree-of-Thoughts (ToT)**	**问题：** 一次生成线性推理受贪婪搜索限制，难进行策略性搜索和回溯 **架构：** 通过将“思考”定义为连贯的中间文本单元，使用树搜索来探索多种推理路径 **关键点：** 模型根据启发式或评分来扩展多个候选“思路”，进行自评并回溯，实现策略性推理和全局决策【61†L27-L35】。	应用于需要规划的任务（24点游戏、创意写作、小型填字等）：GPT-4 CoT在24点游戏上仅解出4%，而ToT方法成功率74%【61†L35-L40】。在Creative Writing等任务上也展现出更高的解决率和多样性。	搜索成本高：需多次评估模型生成，复杂度随搜索宽度和深度指数增长。适用于需要探索性和规划性的任务。提供prompt及代码【61†L37-L40】。
Wang 等 (2024, ICML)【29†L24-L32】 **MemoryLLM**	**问题：** 现有LLM上线后知识静态，难以更新新知识，长期信息保留不足 **架构：** 在Transformer的潜空间引入固定大小记忆池（latent-space memory），模型可写入/更新记忆 **关键点：** 记忆池通过训练学到如何压缩先前信息到潜空间。MemoryLLM能注入文本知识并长期保留，评估表明可持续更新且几乎无性能衰退【29†L24-L32】。	在模型编辑和长期知识保留任务上效果显著。MemoryLLM内部记忆池大小约10亿参数，能有效处理~16K上下文长度内容；在自定义评估和长上下文测试中表现超越无记忆模型【29†L24-L32】。	开源了代码实现【29†L24-L32】。引入大规模记忆池增加训练和推理复杂度，对超长文本（>20K）仍有限制。
Wang 等 (2025, ICML)【28†L23-L31】【28†L32-L36】 **M+ (MemoryLLM+)**	**问题：** MemoryLLM对>20K 长度信息保留不足 **架构：** 在MemoryLLM基础上添加联合训练的检索器和长期记忆机制 **关键点：** 动态地从存储库中检索相关信息辅助生成。MemoryLLM+（M+）有效扩展记忆能力，在保留知识的同时兼顾效率。【28†L23-L31】【28†L32-L36】	在长上下文理解与知识保留基准上，M+大幅超越MemoryLLM和其他强基线：知识保留长度从<20K扩展到>160K，且GPU内存消耗相当【28†L23-L31】【28†L32-L36】。说明引入检索极大提升了长程记忆。	开源代码可用【28†L33-L36】。比原MemoryLLM更耗算力（检索额外计算），但显著扩展了可用序列长度。

方法/论文 (年份/会议)	核心模块与技术细节	数据集与定量结果 (相对基线)	开源代码/复现难度
Li 等 (2025, arXiv)【20†L95-L100】【75†L1-L4】 **START (Self-Taught Reasoner w/ Tools)**	**问题：** 纯内部推理（Long CoT）在复杂计算上易幻觉，缺乏外部辅助【20†L104-L112】；需有效利用工具自我提升 **架构：** 引入工具（代码执行）和自学习框架：1) 提示注入(hint-infer)，在推理生成时插入“建议”信号激励使用工具；2) 受限拒采样Fine-Tuning（Hint-RFT），对Hint-infer生成的带工具调用路径进行筛选并微调。Fine-tune后模型自动学会工具使用。	在PhD级科学QA (GPQA)、AMC/AIME数学竞赛、LiveCodeBench代码竞赛等基准上，START准确率分别达63.6%、95.0%、66.7%、47.1%、47.3%【20†L95-L100】【75†L1-L4】，显著超越基模型QwQ-32B-Preview，并达到当时开源最强模型（R1-Distill-Qwen-32B）和GPT-o1水平。	训练复杂：需提示设计及额外的微调步骤；运行时有工具调用开销。显著提升复杂推理性能，代码/数据集尚未公开，复现难度高。

方法比较分析

从上述表格和工作内容可以看出，长思维链研究呈多维度发展趋势：**提示策略**（CoT、自洽性、最简到复杂、思想树等）旨在通过优化解码或任务分解提高推理能力；**工具集成**（ReAct、Toolformer、PAL等）则通过调用计算器、搜索、编程等外部工具缓解模型自身的计算和知识局限；**记忆增强**（MemoryLLM、M+ 等）通过引入长期记忆模块和检索机制来突破上下文长度限制；**强化自学**（START 等）利用自动提示和强化学习微调进一步提升模型的工具使用和推理质量。具体而言，CoT方式本质上不改变模型结构，其成本最低、易组合，但效果依赖于模型规模和推理任务的连贯性【38†L23-L31】；Self-Consistency 和 ToT 等方法通过多条轨迹搜索及投票极大提升准确率，但计算成本较高，需要进行多次采样和评分【41†L25-L33】【61†L27-L35】。工具集成方法虽然增加了API调用开销，但能显著减少数字计算或事实查询中的错误【48†L33-L40】【53†L29-L37】；其中，PAL 通过程序推理完全规避了基础算术错误【69†L46-L54】【69†L55-L60】，而Toolformer 则让模型自动学习何时调用哪种工具【53†L29-L37】。记忆增强方向下，MemoryLLM通过固定大小记忆池使模型可持续更新知识，M+通过检索器进一步将知识保留扩展到数十万令牌【28†L23-L31】【28†L32-L36】。综合来看，这些方法在**模块类型**（纯生成、工具接口、记忆机制等）、**复杂度**（简单提示 vs. 树搜索 vs. 新模块）和**计算成本**（增益 vs. 重采样 vs. 检索开销）上各有取舍，适用于不同场景。尽管方法繁多，主流思路往往可以组合：例如可将Self-Consistency与ReAct结合，同时采样多条带工具调用的推理路径，也可把ToT与MemoryLLM结合，对候选思路结果进行记忆检索增强。

方法类别	模块类型	时间/空间复杂度 (相对)	可扩展性	计算/内存成本	效果提升	适用序列长度	可组合性
Chain-of-Thought (CoT)【38†L23-L31】	纯提示生成	O(1)（仅提示）	需大型模型	与基础LM相同	依赖规模显著提升	上下文窗口内	高（可与多数策略组合）

方法类别	模块类型	时间/空间复杂度 (相对)	可扩展性	计算/内存成本	效果提升	适用序列长度	可组合性
Self-Consistency 【41†L25-L33】	解码策略（多条采样）	×k（采样倍增）	计算可并行扩展	采样次数多，延迟增加	大幅提升（如算术+17.9%）	窗口内	高（与CoT等组合）
Least-to-Most (LtM) 【44†L25-L34】	任务分解提示	O(n)（子问题链长）	易扩展，提示设计难	需额外示例，提示复杂	显著提升泛化（SCAN+83%）	窗口内	高（与CoT互补）
Tree-of-Thoughts (ToT)【61†L27-L35】	搜索性推理（树状）	指数增长（宽度深度）	受限于搜索策略	极高（多次模型调用）	极大提升规划任务成功率	窗口内	中（需定制化整合）
Tool-Inte. (ReAct, Toolformer)【48†L27-L35】【53†L29-L37】	思考+行动交互；API调用	O(1)+	依赖工具种类/接口	调用工具费用高，延迟	显著减少计算/知识错误	窗口内&可无限（外部）	高（可加在任意CoT基础）
Program-Aided (PAL) 【69†L46-L54】	LLM→代码→执行	O(1)＋执行器	需目标语言支持	执行环境开销	算术性能SOTA（GSM8K+15%）	窗口内&超长（解释器）	中（依赖解释器）
MemoryLLM【29†L24-L32】	Transformer+内生记忆池	O(1) 加密存储	良好（记忆可更新）	额外内存池（1B参）	提升长期保持能力	~16K内有效	高（可与CoT/检索结合）
M+ (Mem. + Retriever) 【28†L23-L31】	内生记忆+检索器	O(1)+检索开销	优（可伸缩检索库）	索引与检索成本	长程保留扩展到160K	~160K	高（结合记忆和检索）

方法类别	模块类型	时间/空间复杂度 (相对)	可扩展性	计算/内存成本	效果提升	适用序列长度	可组合性
Retrieval-Guided (SEEK)【56†L23-L32】	LLM推理草图+结构化检索	O(1)+	良好（检索多轮）	迭代检索成本	提升查询多样性，减少冗余	窗口内+检索结果	中（与CoT/RAG结合）

共性与瓶颈

从现有工作可以看出，许多方法共享如下假设与挑战：**大模型依赖**：多项工作表明，链式推理能力和复杂提示策略往往只有在数百亿参数级别以上模型才能有效【38†L23-L31】；对小模型的改进往往需要额外技术（如知识蒸馏或记忆注入）。**上下文限制**：无论是CoT生成长推理还是RAG检索辅助，序列长度都是瓶颈。Transformer的注意力机制导致长度增长成本激增，且旧信息易遗忘，梯度和注意力随着距离衰减。MemoryLLM一类方法尝试用固定记忆池缓存信息，但在超过约20K令牌时仍会衰减【28†L23-L31】。**推理一致性与错误传播**：长链式推理中，模型初始步骤的微小错误会在后续步骤中放大，缺乏内在纠错机制（ReAct等采用外部资源解决这一点）【48†L36-L43】。此外，CoT生成的过程缺乏明确监督标注，推理路径的正确性很难评估，相关的标注和评测也很难收集。**评估困难**：当前大多关注最终答案准确率，而对中间推理链质量、连贯性或多样性的评估指标较少。**模型假设**：多数方法默认问题具有唯一正确答案、解题步骤可分解且可验证，但现实推理任务（如开放式对话、创作）往往不满足这些假设。此外，不同工作多在特定领域（数学、常识QA、程序题）中验证，很少有统一评价。**常见瓶颈**还包括计算成本高（如ToT的指数搜索开销）、记忆和检索机制调优难度（如何高效检索相关信息）、上下文切分训练策略（如何在不截断信息的情况下训练超长序列）等。总结而言，实现**长程稳定推理**需要同时解决“模型规模依赖”、“序列长度限制”与“推理误差累积”三大瓶颈。

创新机会与实施建议

基于现有成果和未尽之处，我们提出以下潜在突破方向：

1. **记忆增强的链式推理**：将显式长时记忆模块与链式推理相结合，例如在推理过程中将中间步骤编码并存入记忆池，允许模型在后续步骤检索回顾。这可融合MemoryLLM+CoT思路。实施： 设计带记忆读写的推理模型，在生成每一步时同时更新潜在记忆表示；测试模型能否在生成极长推理（例如上千令牌）时保持一致性。挑战： 如何有效索引和检索相关记忆段；避免记忆累积误差；保证模型可微训练。评估： 在Multi-hop QA或多步数学题上，对比有无记忆时的性能与记忆利用率；评估上下文长度扩展效果（如150K vs 20K）。优先级：高。

2. **层次化推理策略**：构建多级推理框架，先进行高层次规划再细化推理，可使模型应对更复杂、长链任务。可参考ToT和LtM思路：外层任务分解（树结构），内层链式生成。实施： 开发双层模型：上层先输出“纲要”或子目标序列，下层根据纲要生成具体推理；或使用RL训练模型学会何时上升/下降层级。挑战： 构造层次化数据或奖励；确保分层推理不丢失信息；合适的搜索策略设计。评估： 在需要分层思考的问题（如复杂故事理解、算法题）上衡量分层策略对准确率和推理长度的提升。优先级：中高。

3. **检索与CoT联合优化**：现有RAG多与生成任务结合，可进一步与CoT深度融合：例如，让CoT提示在检索结果基础上展开推理，或者让检索步骤也服从推理链式计划。实施： 结合SEEK思路，通过提示模型生

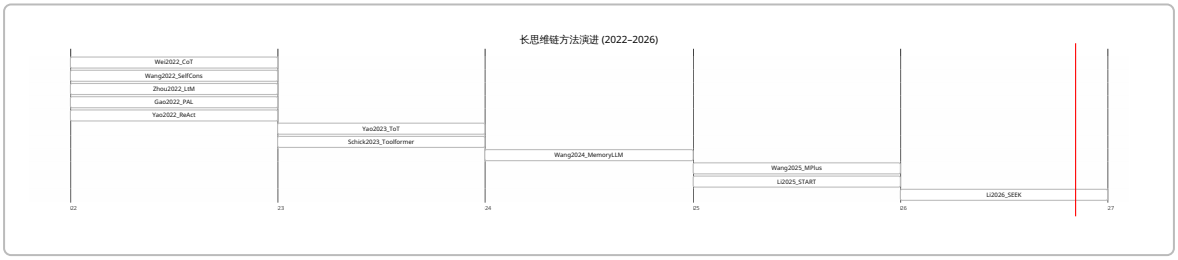
成查询草图，然后检索并将知识填入推理链；或在CoT示例中加入检索交互示例。挑战： 避免检索知识冗余或与内部知识冲突；如何训练模型学会检索时机和问题重写。评估： 在需要外部知识的QA上比较有无检索的CoT性能；度量检索效率（去重率、多样性）。优先级：中。

4. **多模型协同推理**：引入多种专家模型共同解决推理任务，例如一个模型负责生成思路分解、另一个负责具体运算或逻辑检验，还可包含外部符号推理模块。类似于“AI团队”协作。实施： 构建多agent框架，各agent可是不同能力的LLM或神经符号系统，通过交换信息协作推理。挑战： 协调各agent间的信息流；防止信息瓶颈；评估整体增益。评估： 比较联合模型与单一模型在多样化任务（如编程与QA混合）上的表现；分析协同带来的准确率提升。优先级：中低。

5. **自适应推理长度与选择**：开发模型自动决定推理深度和步骤的机制，避免固定长链导致无用或错误信息累积。可采用动态停机策略或推理过程中的不确定性估计。实施： 为模型添加判别器或策略层，根据当前推理置信度决定是否继续展开；或使用RL训练模型优化推理长度。挑战： 设计可靠的停机准则；训练信号匮乏（何时“思考充分”）。评估： 观察在相同任务下，动态推理与固定推理的效率对比；评估推理准确度与步骤数的权衡。优先级：中。

6. **评估和数据集拓展**：构建更全面的标注数据和评估指标，如带有正确推理链的多步推理数据集、推理质量测度（连贯性、一致性）。这虽非算法创新，但对促进算法改进至关重要。实施： 设计标注指南，收集具有中间步骤答案的推理题（数学、逻辑、多语言等）；开发自动评测工具评估推理链质量。挑战： 数据标注费时费力；难以评价“创造性”推理。评估： 数据集多样性和标注质量，及下游模型使用该数据集训练提升情况。优先级：高（基础支撑）。

方法演进图



图：方法演进时间线。2022年以Chain-of-Thought为起点，随后引入自洽性、Least-to-Most、程序辅助（PAL）、ReAct等思路；2023年出现思想树和Toolformer等搜索/工具结合策略；2024–2025年向记忆增强与检索融合（MemoryLLM、M+）拓展；2025年以后出现工具+自学习（START）和结构化检索框架（SEEK）等混合创新。【38†L23-L31】【53†L29-L37】

以上内容和图表综合展示了近年长链式推理技术的核心发展脉络和模块依赖关系。各方法的效果比较如上表所示，尽管研究仍面临挑战，但潜在组合创新方向较多，为后续研究提供了明确思路。各引用均来自原始论文和相应作者发布资料。
